PROJECT-BASED LEARNING

PHASE-III FINAL EVALUATION

Satellite Health Monitoring and Collision Avoidance

Team Sentra — Team ID: DSCPP-III-2026-T004

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team Name: Sentra
Team ID: DSCPP-III-2026-T004
Semester: 3rd
Section: *E,ML2,DS1,H*
Domain: OOPs using C++ / DSA

Team Members

1. Chhavit – 2510011425 (Team Lead)
2. Harsh Jain – 2510370400
3. Satyam Kumar – 2510380092
4. Sudhanshu Ojha – 2510011648

Mentor

Mr. Utkarsh Pant **Interactions:** _____

Project Abstract & Problem Statement

Project Abstract

A C++ simulation of self-managing satellites: they track battery, fuel and software health, find nearby debris with an **AVL tree** of 3D sectors, handle the nearest threat first with a **Min-Heap**, and fire evasive burns. In Phase III it became a **3D website** running on **real CelesTrak data**.

Problem Statement

Earth's orbit is filling with debris. Satellites need to dodge it and fix their own faults automatically, without waiting for slow commands from the ground.

Target Users / Application

- Students and teachers learning orbital safety, data structures and OOP
- Exploring real debris clouds (e.g. Fengyun-1C) around real satellites

Objectives & Final Deliverables

Finalized Objectives

1. Find nearby debris fast with an AVL tree of 3D sectors
2. Handle the nearest threat first with a Min-Heap
3. Keep satellites healthy: fuel, battery, self-repair, safe mode
4. Log every event and position (`blackbox.txt`, `telemetry.csv`)
5. Show the simulation in 3D on real CelesTrak data

Final Deliverables

- C++ simulation: `orbital.cpp` + 5 header files
- Interactive 3D website (also one offline `orbital.html`)
- Real data: CelesTrak download scripts (6 groups)
- In-app "How Orbital works" guide
- Phase III report and licence

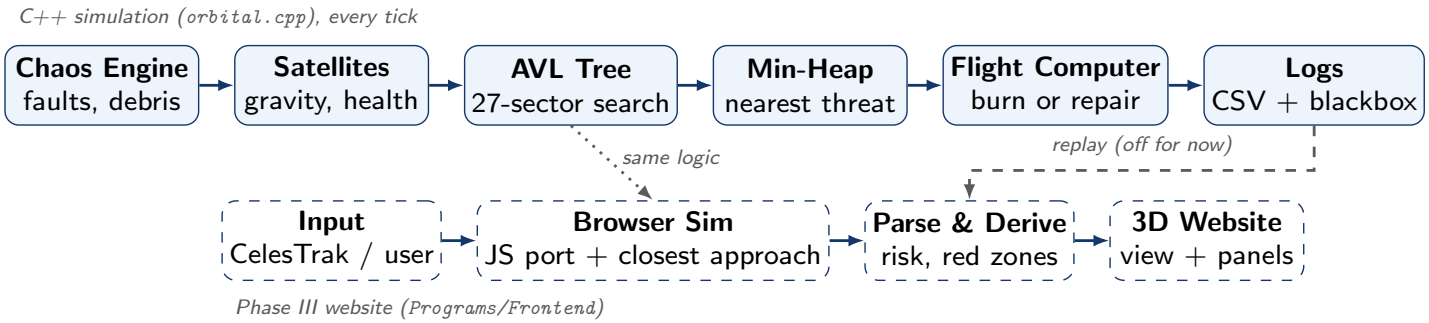
Phase-I & Phase-II Feedback Incorporated

Phase	Feedback / Suggestion	Action Taken
Phase-I	Working in one big file may cause Git merge conflicts.	Divided the code into 5 header files, one per module.
Phase-I	Program crashes if the user types letters instead of numbers.	Added strict `cin` validation loops.
Phase-I	Satellites move in a straight line instead of an orbit.	Added gravity so satellites follow circular orbits.
Phase-II	How will real data be used?	Real satellites and debris load from CelesTrak CSV files in the website; C++ real-data mode in progress.
Phase-II	How is the red zone area defined?	A sector cube holding 2+ debris, shown as a red box in 3D; satellites entering it divert (1.5 kg fuel).
Phase-II	What about satellite-to-satellite collision?	Satellites also check each other: SAT-WARNING, SAT-COLLISION ALERT and evasive burn; docked modules joined.

Final Improvement

From a terminal-only C++ program (Phase I–II) to a 3D website running the same logic on real satellites and debris, with alerts checked between ticks so fast debris is no longer missed.

Final System Architecture



Major Components

- C++: `orbital.cpp` + 5 headers (OOP & DSA)
- Website: `Programs/Frontend` (HTML, CSS, `js/`)

System Flow

- Input: CelesTrak CSV or typed satellites
- Each tick: move, scan, decide, log → 3D view

Final Technology Stack

Software / Technologies

- Programming: C++, HTML, CSS, JavaScript
- Framework: None (plain JavaScript)
- Database: None (CSV and text files)
- Libraries/APIs: `<cmath>`, `<fstream>`, HTML Canvas
- OS/Platform: Windows, macOS, Linux; any browser

Development Infrastructure

- IDE: *Visual Studio Code*
- Version Control: Git/GitHub
- Testing Tools: Headless Chromium, Node.js
- Deployment: Offline single HTML file
- Other Tools: CelesTrak data, `curl` scripts

Change from Phase II: The terminal interface was replaced by a web front end, as planned. AI assistance was used for the website.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	AVL tree with rotations (3D sector search), Min-Heap priority queue (nearest threat first), Circular Queue (battery history), Linear Search (find satellite by name)	`AVL_Tree.h`, `Queues_DSA.h`, `orbital.cpp`
OOPs with C++	Abstract class, inheritance, polymorphism (virtual `runDiagnostics`), function overloading (`handleCollisionThreat`), operator overloading (`Vector3`), templates, file handling, manual memory management	`Satellites_OOP.h`, `FlightComputer.h`, `Queues_DSA.h`

Module-Wise Final Implementation

Module	Implementation	Responsible	Status
`Satellites_OOP.h`	Satellite classes, fuel use, nearest-satellite tracking	Chhavit	Completed
`Chaos_Engine.h`, `FlightComputer.h`	Faults, new debris; risk levels, burns, repairs	Harsh Jain	Completed
`AVL_Tree.h`	3D sector index with rotations, gravity	Satyam Kumar	Completed
`Queues_DSA.h`	Min-Heap of threats, Circular Queue, input checks	Sudhanshu Ojha	Completed
Website: page, simulator, import	Form and guide; browser simulator, CelesTrak import	All members	Completed
Website: 3D view (`render.js`)	Canvas 2D projection, Earth, orbit paths, camera views	Chhavit & Satyam	Completed
Website: playback, panels	Timeline, Burn focus, panels, raw data table	Harsh & Sudhanshu	Completed
C++ real-data mode	Kilometre units, bug fixes, closest approach	*Member(s)*	In progress

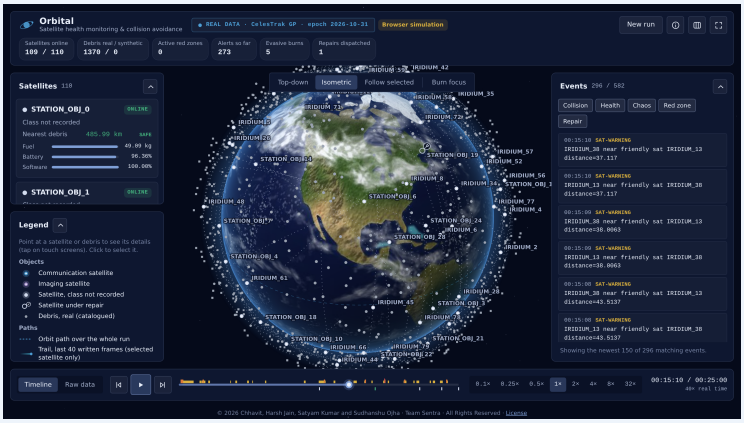
Final Project Completion: *95-98%*

Working System / Prototype Demonstration

Major Working Features

- Real satellites and debris from CelesTrak in 3D
- Warnings, collision alerts and evasive burns
- Burn focus: slows down and zooms in on each burn
- Health: battery, fuel, software; chaos faults; repairs
- Timeline, event feed, raw data, per-satellite charts

System Screenshot / Output



Data / Database / Backend Implementation

Data / Database

- Source: CelesTrak orbital data (CSV files)
- `telemetry.csv`: tick, type, name, X, Y, Z
- `blackbox.txt`: every event, in order
- In memory: AVL tree, Min-Heap, Circular Queue
- Checks: `cin` loops; CSV header, orbit and duplicate checks

Backend / Core Logic

- Tick loop: chaos, move, scan, decide, log
- No server or API: files read locally
- 27-sector AVL search; closest approach between ticks
- No accounts; nothing is uploaded
- Clear messages for wrong files and oversized runs

Testing & Validation

Test Type	Expected Result	Observed Result	Status
Closest-approach check	Correct closest distance in 4 geometric cases	All 4 cases match	Pass
Docked-module joining	ISS modules joined into one spacecraft	131,443 false alerts reduced to 0	Pass
Log event synchronisation	Every event belongs to a known satellite	All events matched and on the timeline	Pass
Large input and speed	180 satellites, 4,070 debris run smoothly	Burn focus 2.5 $\rightarrow$ 38 frames/s	Pass
C++ code review	Unique sector keys; one burn per threat	Key clash at index 500; fuel every tick	Fail

Validation Summary

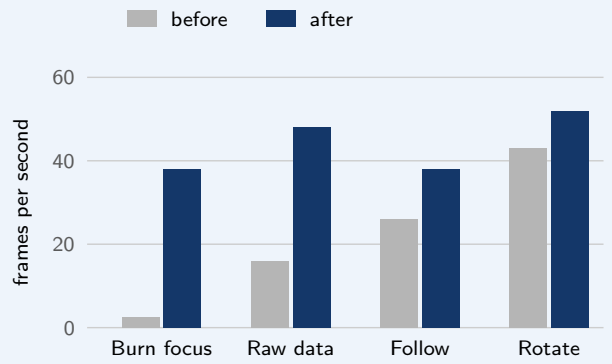
Browser tests ran on real CelesTrak data; Phase II C++ tests passed. The C++ bugs found will be fixed.

Results & Performance Analysis

Key Results

- Closest-approach check: debris collision alerts 1 → 3 on the same run
- Docked modules (POISK on ISS): 131,443 false alerts → 0
- 180 satellites + 4,070 debris simulated in about 16 s
- Close-up burn view: 2.5 → 38 frames per second

Graph / Result Evidence



Website speed before and after optimisation (180 satellites, 4,070 debris).

GitHub Repository & Development Evidence

Repository Details

- Repository: `github.com/Chhavit09/SHMCA`
- Branch: `main`
- Website: `Programs/Frontend`
- README.md: *Updated*
- Contributors: all four members

Phase III Commits

- `2cd7b17` Core, parsers, simulator (harshj1214-hj)
- `c5b8648` Page, New run form, guide (Chhavit09)
- `6309fc3` 3D projection, Earth, camera (kr-Satyam09)
- `9d4cd9e` Playback, Burn focus (harshj1214-hj)
- `c44cc37` Panels, raw data table (sudhanshuOjha07)

Each commit also lists co-authors; merged into `main` through a reviewed pull request.

Individual Contribution

Member	Role	Final Technical Contribution	Commits
Chhavit	Team Lead	`Satellites_OOP.h`; website page design, New run form and guide; 3D view with Satyam	14*
Harsh Jain	DSA/Sim Dev	`Chaos_Engine.h`, `FlightComputer.h`; browser simulator and parsers; playback and Burn focus	21*
Satyam Kumar	DSA Dev	`AVL_Tree.h`, vector maths, gravity; 3D projection, Earth and camera views	21*
Sudhanshu Ojha	DSA Dev	`Queues_DSA.h` (Min-Heap, Circular Queue), input checks; panels, event feed, raw data table	10*

* Commits on `main` by main author after the Phase III pull request is merged; check the PBL portal.

Individual Evaluation

Every member can explain, demonstrate and defend their own module and its data structures.

Challenges, Limitations & Lessons Learned

Challenges & Solutions

- Fast debris missed between ticks – closest-approach check
- Docked modules flooding alerts – join them as one spacecraft
- 3.9 GB telemetry – write every N ticks, size check

Current Limitations

- C++ still classic units only; website runs a JavaScript copy
- No return to orbit after a burn; red zones never expire

Technical Learning

- Choosing data structures for spatial search
- Discrete time steps can miss fast events
- Real data breaks assumptions that test data hides

Team Learning

- Separate header files ended merge conflicts
- Measure first, then optimise

Final Outcomes, Applications & Future Scope

Final Outcomes

- Working C++ simulation
- 3D website on real data
- Measured, documented fixes

Applications

- Teaching DSA and OOP
- Explaining debris risk
- Testing avoidance rules

Future Scope

- C++ real-data mode and fixes
- Return-to-orbit burns, SGP4
- AVL vs linear scan study

Tangible Project Outcome

A working prototype: a C++ simulation and a 3D website that runs offline as a single HTML file, simulating real satellites and debris from CelesTrak with every alert, burn and repair visible.

References & Project Resources

References

1. D. J. Kessler, B. G. Cour-Palais, "Collision frequency of artificial satellites: the creation of a debris belt," *J. Geophys. Res.*, 1978.
2. F. R. Hoots, R. L. Roehrich, *Spacetrack Report No. 3: Models for Propagation of NORAD Element Sets*, 1980.
3. D. A. Vallado, *Fundamentals of Astrodynamics and Applications*.
4. T. H. Cormen et al., *Introduction to Algorithms* (AVL trees, heaps).
5. CelesTrak GP data, `celestrak.org`.
6. A Causal Graph-Based Framework for Satellite Health Monitoring (Meng et al., 2023).
7. Risk Assessment of Satellite Constellations Using Network Theory (Lin et al., 2024).

Project Resources

- GitHub: `github.com/Chhavit09/SHMCA`
- Demo: `Programs/Frontend/index.html`
- Dataset: CelesTrak (`celestrak.org`)
- Documentation: in-app guide

Thank You

Questions & Discussion